

Prompt Engineering

Mehmet Ali Akyol, PhD

Ankara Medipol University

December 19, 2025

Week 10: Prompting for Complex Problem Solving

Weekly Learning Objectives

- **Decomposition:** Break down ambiguous problems into promptable components.
- **Reasoning:** Facilitate reasoning chains (CoT), scenario analysis, and simulation.
- **Augmentation:** Integrate external knowledge bases and tools (Search, Code).
- **Validation:** Manage uncertainty with hypotheses, assumptions, and validation.

From Tasks to Problems

The Shift

In previous weeks, we focused on **Tasks**: discrete, well-defined actions (e.g., "Summarize this," "Write code for X").

- **Simple Tasks:** Linear, one right answer, low ambiguity.
- **Complex Problems:** Non-linear, multiple valid solutions, high ambiguity (e.g., "How do we reduce customer churn?").
- **The Challenge:** LLMs struggle with long-horizon planning without guidance.
- **The Solution:** We must act as **Architects**, breaking the problem down into manageable "promptable" chunks.

The Cognitive Architecture of LLMs

Why do they struggle with planning?

LLMs are probabilistic next-token predictors, not logical planners with an internal world model.

- **No Internal State:** They don't "remember" the plan unless it's in the context window.
- **Greedy Decoding:** They often pick the most likely *next* word, not the best word for a goal 10 steps away.
- **Hallucination Accumulation:** One small error in step 1 compounds by step 10.

Implication: We must externalize the "state" and "plan" into the prompt itself.

Key Concepts

- **MECE Principle:** "Mutually Exclusive, Collectively Exhaustive." A way to group information so nothing overlaps and nothing is missing.
- **Decomposition:** The art of slicing a big problem into small tasks.
- **Chain-of-Thought (CoT):** Prompting the model to "think out loud" before answering.
- **Tree of Thoughts (ToT):** Exploring multiple possible reasoning paths simultaneously.
- **Self-Consistency:** Asking the model the same question multiple times to find the most stable answer.

Mini example (MECE)

Problem: "Improve university cafeteria experience". MECE buckets: *Food quality* vs. *Service speed* vs. *Price* vs. *Seating/cleanliness*.

Clarifying the Problem

Why it matters

If you ask a vague question, you get a vague answer. Complex problems need strict boundaries.

- **Scope:** What is IN and what is OUT?
- **Constraints:** Budget, time, technology, legal.
- **Success Metrics:** How do we know we solved it?

Tip: Ask the model to interview *you* before it starts solving. *"Before you answer, ask me 3 clarifying questions to better understand the context."*

Defining Success Criteria

- **Qualitative Metrics:** Tone, clarity, adherence to brand voice.
- **Quantitative Metrics:** Length (words/lines), complexity (reading level), format (JSON schema).
- **Negative Constraints:** What should the model *avoid*? (e.g., "Do not use jargon," "Do not assume unlimited budget").

Prompt Pattern

"Your solution will be evaluated on: 1) Feasibility (can we do it in 2 weeks?), 2) Cost (under \$100), 3) Impact (must touch 50% of users)."

Framework: SCQA

- **Situation:** The stable context. ("We are a B2B SaaS company.")
- **Complication:** The change or problem. ("Churn increased by 5% after the last UI update.")
- **Question:** The core problem to solve. ("Is the UI the cause, or is it coincidence? How do we fix it?")
- **Answer:** The proposed solution (The LLM's job).

Example Prompt

"Context: [Situation]. Problem: [Complication]. Core Question: [Question]. Propose 3 distinct strategies."

Decomposition Strategies

- **Sequential Decomposition:** Step B depends on Step A.
 - *Example:* Research Topic → Outline Essay → Write Draft.
- **Parallel Decomposition:** Steps can happen at the same time.
- **Least-to-Most Prompting:**
 - 1 Decompose problem into sub-questions.
 - 2 Solve the simplest sub-question.
 - 3 Use that answer to solve the next.

Least-to-Most Example

Q: "Last letter of the name of the CEO of the company that made the iPhone?" 1. Who made the iPhone? (Apple) 2. Who is the CEO of Apple? (Tim Cook) 3. Last letter of Tim Cook? (k)

Recursive Decomposition

For Massive Problems

Sometimes a sub-task is still too big. You must recurse.

- **Level 1:** Build a Mobile App.
- **Level 2 (Decomposed):** Backend, Frontend, Design.
- **Level 3 (Recursive - Backend):** Database Schema, API Endpoints, Auth System.
- **Level 4 (Recursive - Auth):** Login, Signup, Password Reset.

Prompting Tip: "Break this task into 3 sub-tasks. Then, for each sub-task, break it down further into actionable steps."

Decomposition Example: Reduce Customer Churn

Start with a vague problem

"How do we reduce customer churn?"

- **Clarify:** What is churn (monthly? yearly?), which customer segment, what timeframe?
- **Break into sub-problems:**
 - Diagnose *why* people leave (pricing, product bugs, competitors).
 - Identify *where* churn is highest (region, device, plan).
 - Propose interventions (onboarding, support, discounts).
 - Define measurement (A/B test, retention cohort).
- **Integration:** Combine results into one plan with priorities and risks.

Chain-of-Thought (CoT)

The Concept

LLMs are probabilistic. If you ask for the answer immediately, they guess. If you ask them to "show their work," they reason.

Zero-Shot CoT

- Just add magic words.
- "Let's think step by step."
- Good for general logic.

Few-Shot CoT

- Provide examples of reasoning.
- Q: ... A: Step 1... Step 2... Answer.
- Better for specific formats.

Why it works: It forces the model to generate more tokens (computation) before committing to a final answer.

Tree of Thoughts (ToT)

- **Linear Reasoning:** $A \rightarrow B \rightarrow C$. (If A is wrong, everything is wrong).
- **Tree Reasoning:** Explore multiple branches.

How to Prompt ToT

- 1 **Brainstorm:** "Generate 3 distinct solutions to this problem."
- 2 **Evaluate:** "For each solution, list pros, cons, and probability of success."
- 3 **Select:** "Discard the worst two. Pick the best one and refine it."

Bridging Reasoning and Tools

Reasoning alone is hallucination-prone. Action alone is unguided. ReAct combines them.

- **Thought:** The model analyzes the current situation. ("I need to find the CEO's age.")
- **Action:** The model decides to use a tool. ("Search: CEO of Apple age")
- **Observation:** The tool returns data. ("Tim Cook is 63.")
- **Thought:** The model processes the data. ("Now I can answer the question.")

Self-Consistency (Simple Checking)

Idea

If one answer might be a guess, generate **multiple independent answers** and pick the most consistent one.

- Ask the model: "Answer the question 3 times. Each time use a different reasoning path."
- Compare answers. If they disagree, ask: "What assumption caused the difference?"
- Useful for: estimation, ambiguous reasoning, and planning.

Mini example

"How long will a student take to learn Python basics?" → 3 estimates + assumptions (hours/week, prior experience).

Debate and Critique (Finding Blind Spots)

- **Debate:** Two personas argue different choices (e.g., "Build it" vs. "Buy it").
- **Critique:** A reviewer checks logic: missing data, unrealistic assumptions, risks.
- **Moderator:** Summarizes consensus + open questions.

Example prompt

"Persona A argues for option 1, Persona B argues for option 2. Then a Moderator lists: pros/cons, key assumptions, what data would decide."

Why do we need tools?

- **Math:** LLMs are "calculators for words," not numbers. They often fail at arithmetic (e.g., 234×892).
- **Current Events:** Models have a "Knowledge Cutoff" (e.g., 2023). They don't know today's stock price.
- **Private Data:** They don't know your company's internal documents.

Solution: Give the model "hands" (Tools) and "eyes" (Retrieval).

Retrieval Augmented Generation (RAG)

The "Open Book Exam" Analogy

Instead of asking the model to memorize everything (Closed Book), we give it the textbook (Context) and ask it to find the answer.

- **Step 1 (Chunking):** Break documents into small pieces (paragraphs).
- **Step 2 (Retrieval):** Search your database for relevant text based on the user's question.
- **Step 3 (Augment):** Paste that text into the prompt.
- **Step 4 (Generate):** "Answer the user's question using ONLY the context above."

Benefit: Drastically reduces hallucinations and enables fact-checking.

Function Calling (Tool Use)

How it works

You don't run the tool; the model *asks* you to run it.

- 1 **User:** "What's the weather in Ankara?"
- 2 **Model (Thought):** I can't know this. I have a tool `get_weather(city)`.
- 3 **Model (Output):** CALL: `get_weather("Ankara")`
- 4 **System:** Runs the code, gets "25C, Sunny".
- 5 **Model (Final):** "It is 25 degrees and sunny in Ankara."

Tool-Use Prompt Template

Role: You are a careful assistant.

Task: Solve the problem.

Tools Available: [Calculator, Google Search, Python Interpreter]

Rules:

- 1) If calculation is needed, say: "I will use a calculator" and
- 2) If a fact might be outdated, say: "I need to search" and c
- 3) If you lack information, ask 2 clarifying questions.

Output: Final answer + short assumptions list.

Validation: How to reduce mistakes

- **Check constraints:** Does the solution respect budget/time/legal limits?
- **Ask for edge cases:** "What could go wrong?" / "Worst-case scenario?"
- **Ask for tests:** For code, request test cases; for plans, request success metrics.
- **Be honest about uncertainty:** If the model is guessing, it should say so.

Adversarial Prompting (Red Teaming)

Playing Devil's Advocate

Ask the model to critique its own work to find flaws.

- **Prompt:** "You are a harsh critic. Review the plan above. Find 3 logical gaps, 2 risky assumptions, and 1 safety violation."
- **Then:** "Now, act as the Architect again. Fix the plan based on the critic's feedback."

Example 1: The Marketing Plan (Iterative Refinement)

The Scenario

You are opening a new coffee shop in a university town with a low budget.

Bad Prompt: "Write a marketing plan for my coffee shop."

Better Prompt (Context): "We are opening near a university. Budget \$500/month. Target: Students."

Best Prompt (Architected):

- **Role:** "Act as a Senior Marketing Strategist."
- **Task:** "Create a 4-week launch plan."
- **Constraints:** "Focus on viral social media and campus partnerships. No paid ads."
- **Output:** "Table format: Week | Channel | Activity | KPI | Estimated Cost."
- **Validation:** "After the table, list 3 risks that could make this plan fail."

Example 2: The Coding Architect

The Scenario

You need a Python script to scrape a website, clean data, and save to CSV.

Step 1 (Design): "Act as a System Architect. Don't write code yet. Outline the structure for a Python scraper. What libraries should we use? How should we handle errors?"

Step 2 (Implementation): "Great plan. Now, write the code for the `fetch_data` function using `requests` and `BeautifulSoup` as you proposed. Include error handling for 404 and 500 errors."

Step 3 (Test): "Write a unit test for this function using `pytest`. Mock the network response."

Example 3: Explaining Complex Topics

The Scenario

You need to explain Quantum Computing to a non-technical audience.

Technique: **Analogy + Simplification**

Prompt: "Explain Quantum Computing to a 10-year-old.

- Use the analogy of a coin spinning on a table (vs. a coin lying flat).
- Avoid jargon like 'superposition' or 'entanglement' unless you define them simply.
- Focus on why it is faster than normal computers."

Example 3: Debugging a Program (Structured Prompt)

The Scenario

A student has a Python error but doesn't know where it comes from.

Prompt pattern: Give code + error + what you tried, then ask for a plan.

- **Context:** "I am using Python 3.11 and VS Code."
- **Input:** Paste the error message and the minimal code snippet.
- **Task:** "Explain the error in simple terms, propose 3 likely causes, and give step-by-step fixes."
- **Output:** "Use a numbered list. Include a corrected code block."

Key Takeaways

- **Don't be lazy:** Complex problems require complex prompts.
- **Break it down:** If a problem is too big, split it into smaller pieces (Decomposition).
- **Ask for logic:** Use "Let's think step by step" (Chain-of-Thought) to reduce errors.
- **Give examples:** Show the model what you want (Few-Shot).

Bonus takeaway

Always add a **validation step**: ask for assumptions, risks, and how to test the solution.

Thank You!

Dr. Mehmet Ali Akyol

mehmetali.akyol@gmail.com

Questions?